

Trackzio Movie Discovery App

Full-Stack Intern Assignment Documentation

Project Summary

A responsive full-stack movie discovery application built with React, Node.js, Express, MongoDB, and the TMDB API. The product supports discovery, search, genre exploration, sorting, pagination, movie details, and persistent wishlist management.

Core Technologies

Frontend	Backend	Database	External API
React + Vite React Router Axios + CSS	Node.js + Express Axios + Mongoose REST API	MongoDB Atlas Wishlist schema	TMDB Movie API

Prepared for submission as supporting documentation. Live deployment and repository links can be added after hosting is finalized.

1. Product Overview

The application is designed as a movie discovery product rather than a simple third-party API demo. Users can enter the product on a discovery page, browse popular movies immediately, explore categories, search when needed, change result ordering, open a dedicated details view, and maintain a persistent wishlist.

The frontend communicates only with the Node.js backend. The backend owns external TMDB communication, response transformation, error handling, and a short-lived in-memory cache. Wishlist records are stored in MongoDB so saved movies remain available after the browser is reopened.

Key assignment-aligned capabilities

- Browse without searching first through popular movie results.
- Search movies using the backend abstraction layer.
- Explore by genre and continue with pagination.
- Sort loaded results by rating, title, and release date.
- Open movie details without losing the application's navigation structure.
- Persist wishlist data in MongoDB and prevent duplicate entries.
- Show loading, empty, retry, and error feedback for common failure cases.
- Use lazy-loaded posters and responsive layouts for different screen widths.

2. Architecture and Data Flow

The project uses a layered architecture to keep responsibilities separate and make the application easier to explain, test, and extend.

```
React Frontend
|
| REST API calls
v
Node.js + Express
|
+-----+
| |
v v
TMDB Service MongoDB
|
v
TMDB API
```

Frontend responsibilities

- Render discovery, search, genre, sorting, details, and wishlist views.
- Maintain UI state such as current page, selected genre, search query, loading state, and wishlist IDs.
- Navigate between pages with React Router.
- Call backend endpoints with Axios rather than calling TMDB directly.

Backend responsibilities

- Expose REST endpoints for movies, search, genres, details, and wishlist operations.
- Validate basic request inputs and return application-level errors.
- Transform external TMDB responses into smaller, frontend-focused movie objects.
- Centralize TMDB access inside a service layer and cache repeated requests for five minutes.

3. Project Structure

```
trackzio-movie-app/
|-- README.md
|-- .gitignore
|
|-- frontend/
|   |-- src/
|   |   |-- App.jsx
|   |   |-- App.css
|   |   |-- MovieDetails.jsx
|   |   |-- Wishlist.jsx
|   |   |-- main.jsx
|   |   |-- index.css
|   |-- .env.example
|   |-- package.json
|
|-- backend/
|-- src/
|   |-- controllers/
|   |   |-- movieController.js
|   |-- models/
|   |   |-- Wishlist.js
|   |-- routes/
|   |   |-- movieRoutes.js
|   |   |-- wishlistRoutes.js
|   |-- services/
|   |   |-- tmdbService.js
|   |-- server.js
|-- .gitignore
|-- package.json
```

4. Backend API

Method	Endpoint	Purpose
GET	/api/movies	Popular movies, optionally filtered by genre and page.
GET	/api/movies/search?q=...	Search movies with pagination.
GET	/api/movies/:id	Retrieve details for one movie.
GET	/api/movies/genres	Retrieve TMDB movie genres.
GET	/api/wishlist	Read persisted wishlist entries.
POST	/api/wishlist	Create a wishlist movie; duplicate movie IDs return a conflict.
DELETE	/api/wishlist/:movieId	Remove a wishlist movie.

The movie endpoints return normalized objects such as id, title, poster, rating, releaseDate, overview, and genres so the frontend does not depend on every field in the raw TMDB payload.

5. Wishlist Data Model

```
{
  movieId: Number,
  title: String,
  poster: String | null,
  rating: Number,
  releaseDate: String,
  overview: String,
  createdAt: Date,
  updatedAt: Date
}
```

The wishlist schema uses a unique movieId so the same movie cannot be added twice. Only the data required to render and manage the saved movie is stored; full external movie records are not copied into MongoDB.

6. Key Technical Decisions

Backend abstraction

TMDB is accessed from Node.js rather than directly from the client. This centralizes API credentials, response shaping, caching, and error handling.

Pagination

Movie pages are fetched incrementally so large result sets are not requested all at once. A Load More action allows users to continue exploring.

Client-side sorting

Sorting is applied to the movies already loaded in the client, avoiding additional external requests for each sort change.

In-memory cache

Identical TMDB endpoint and parameter combinations are cached for five minutes to reduce repeated external calls during a session.

Environment variables

The frontend uses VITE_API_URL and the backend keeps TMDB and MongoDB secrets in environment variables rather than source code.

Persistent wishlist

Wishlist state is stored in MongoDB instead of only in browser memory, so saved items survive application restarts.

7. Error Handling and Edge Cases

- No search results: the UI shows a clear no-results state instead of an empty grid.
- Invalid movie ID: the details view returns a friendly not-found message and a navigation path back to Discover.
- Backend or external API failure: the UI shows an error state with a retry action where appropriate.
- TMDB rate limiting: the backend recognizes status 429 and returns a specific rate-limit message.
- Missing poster: the frontend renders a placeholder instead of a broken image.
- Duplicate wishlist entry: MongoDB uniqueness plus backend validation prevents duplicate movie IDs.
- Rapid category interactions: the UI keeps the selected state aligned with the latest category action.

- Large result sets: pagination and Load More avoid rendering every possible result at once.

8. Final Testing Checklist

Area	Verification
Discover	Popular movies load without an initial search; movie cards and navigation work.
Search	Valid searches return results; unlikely queries return a no-results state.
Genres	Genre selection changes the result set; Load More continues within the selected genre.
Sorting	All configured sorting options reorder the currently loaded movies.
Details	Movie detail pages load metadata, genres, overview, and wishlist action.
Wishlist	Add, remove, duplicate prevention, and persistence after reopening all work.
Failure handling	Stopping the backend surfaces an error instead of a blank page; restarting restores functionality.
Responsive	Desktop, tablet, and mobile layouts were checked for overflow and overlap.

9. Local Setup

Prerequisites: Node.js, npm, Git, a MongoDB Atlas database, and a TMDB developer account.

Backend

```
cd backend
npm install

# backend/.env
TMDB_ACCESS_TOKEN=your_tmdb_access_token
MONGODB_URI=your_mongodb_connection_string
PORT=5000

npm run dev
```

Frontend

```
cd frontend
npm install

# frontend/.env
VITE_API_URL=http://localhost:5000

npm run dev
```

For production deployment, the same environment variables should be added in the hosting platform rather than committed to source control.

10. AI Usage and Development Transparency

AI tools were used as development assistance during the project for activities such as understanding API documentation, exploring implementation approaches, generating boilerplate, troubleshooting

development issues, reviewing code structure, and improving error handling.

The project architecture, technology selection, data flow, database persistence approach, feature behavior, and final implementation decisions were reviewed and adapted for the assignment. The developer is responsible for understanding the implemented code and being able to explain, debug, modify, and extend it.

This documentation is intended to accompany the source repository and help a reviewer quickly understand the product flow, technical architecture, persistence model, performance considerations, and testing coverage.

Submission Notes

Before final submission, add the GitHub repository link and any deployed frontend/backend URLs to the repository README. Keep real TMDB access tokens and MongoDB credentials out of Git history and source files.

Recommended attachment filename: Trackzio_Movie_Discovery_App_Documentation.pdf